HTL Leonding
next level

HÖHERE TECHNISCHE
BUNDESLEHRANSTALT
LEONDING

Script Assembler – Generation of Test Scripts for KeTop Devices

DIPLOMA THESIS

submitted for the

Reife- und Diplomprüfung

at the

Department of Informatik

Submitted by:
Filip Schauer
Axel Csomány
Adam Höllerl

Supervisor:
Adrian Kern

Project Partner:
Martin Wieser, KEBA Group AG

Leonding, April 7, 2026

I hereby declare that I have composed the presented paper independently and on my own, without any other resources than the ones indicated. All thoughts taken directly or indirectly from external sources are properly denoted as such.

This paper has neither been previously submitted to another authority nor has it been published yet.

The presented paper is identical to the electronically transmitted document.

Leonding, April 7, 2026 Filip Schauer , Axel Csomány & Adam Höllerl

Abstract

Brief summary of our amazing work. In English. This is the only time we have to include a picture within the text. The picture should somehow represent your thesis. This is untypical for scientific work but required by the powers that are. Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Zusammenfassung

Zusammenfassung unserer genialen Arbeit. Auf Deutsch. Das ist das einzige Mal, dass eine Grafik in den Textfluss eingebunden wird. Die gewählte Grafik soll irgendwie eure Arbeit repräsentieren. Das ist ungewöhnlich für eine wissenschaftliche Arbeit aber eine Anforderung der Obrigkeit. *Bitte auf keinen Fall mit der Zusammenfassung verwechseln, die den Abschluss der Arbeit bildet!* Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Contents

Part I.

Introduction [X, F]

1. Background [X]

1.1. Testing in Industrial Systems

The testing of systems is a crucial part of the development process. It not only ensures that the system functions as intended but also helps identify flaws in areas not considered during the design phase. Testing can be performed at various stages of development, from unit testing to integration testing and system testing.

In the context of industrial automation, testing is particularly important due to the complexity and critical nature of the components involved. The systems in question often control machinery and processes that can bring about significan consequences if they fail. Therefore, a robust testing strategy is essential in order to ensure the reliability of these systems.

Companies operating in this field must develop and maintain thorough testing strategies to ensure system reliability, safety and with that the quality of their products and the satisfaction of their customers. This project focuses on improving the existing testing systems of the KEBA Group AG.

1.2. Company Overview: KEBA Group AG

KEBA Group AG is an austrian technology company specializing in automation systems for industry, banking and energy applications. It designs and produces hardware and software solutions such as machine and robot controls, automated handover terminals, and electric vehicle charging stations.

For more than 50 years, they have been providing various customers with solutions to their unique problems. Among its three main branches, the Industrial Automation

branch focuses primarily on the needs of the individual situation as the hardware and its software get built.

The company was founded in 1968. Today with 28 establishments in 16 different countries and more than 2000 employees they are internationally active, serving global markets from its headquarters in Linz, Austria.

Within the industrial context, this project was conducted in collaboration with KEBA Group AG.

2. Initial Problem [X]

2.1. Current Testing Strategy and its Flaws

So called "KeTops" are used as operating devices to control machines of varying structure and complexity. These machines and their corresponding KeTops differ in their configuration from each customer which is why fitting test scripts are required. These test scripts examine all aspects of the device from the installed operating system to software specific behaviors.

The "Script Assembler" which is used in current production is able to merge a number of test scripts together for this use. The hard-coded nature and resulting limited maintainibility of the Script Assembler is its primary flaw. Furthermore all data concerning tests are saved and maintained in a TestRail database which although functional, heavily limits the expandability and independence of the testing environment.

2.2. Goals and Objectives

This project aims to eliminate all of the listed problems and concerns while building upon and improving the existing capabilites of the Script Assembler.

- **EBNF parsing:**
  The existing structure of the test scripts should be formed into its own EBNF language which is then integrated into the Script Assembler. This eliminates a majority of the hard-coded parts of the software. Furthermore the editor should provide appropriate syntax highlighting when writing or editing a test script.
- **Keyword support:**
  New functionalities should be implemented which allows the editor of test scripts

to specify aspects like the version and the operating system the test has to be run on. The structure of keywords need to be precisely set in the EBNF language.

- **Database mock:**
  The structure of the currently used TestRail database should be studied and an accurate mockup should be created. A seamless switch between the mock and the real TestRail instance has to be implemented as well.

3. Used Technologies [F]

Just like the TRP project for which this project aims to provite an extension for, it was mainly written in the Python programming language. Aside from that, other tools have also been used or are otherwise important to be mentioned. The goal of this section is to give a broad overview of some of the most important technologies that each make up an integral part of our final project.

3.1. Python

Python is a high-level, interpreted, dynamically typed object oriented programming language with a focus on allowing programmers to create applications, scripts, or connections between existing programs, quickly.[1]

3.1.1. Modules

Importantly, Python allows for the import of "modules", which are essentially files containing Python code which allow for importing functions, classes, or similar, into other Python files.[2]

While it is possible for modules to be local files, they are also commonly downloaded from the internet using "package managers". An commonly used example of a package manager for Python, which is also the only Python package manager used for this project, is known as `pip`.[3]

The external modules used in this project include, but are not limited to:

- `lark`, which allows for parsing of a given piece of text using a custom grammar language which is based on EBNF. The core of our parser is built around this library.[4]

- `semantic-version`, which is a small library that provides tools to handle "SemVer".[5] "SemVer" stands for "Semantic Versioning", and it refers to a scheme that intents to dictate how a software's version number is to be assigned and incremented.[6]. This library is used to for parsing version strings for our "`@VERSION` Keyword".
- `psycopg`, which allows for communication with a "PostgreSQL" database.[7] This library is used for our "Mockup Database" system.

3.2. Robot Framework

Robot Framework is a framework for test automation. It allows for defining so-called "Test Suites", which are files ending in `.robot` and may contain any amount of test cases.[8]

In the context of KEBA Group AG, Robot Framework is used to execute various test cases on KeTop devices. However, since our parser only acts as a preprocessor to robot script files, treating them as arbitrary text to be either included, excluded, and/or moved to different positions depending on the defined keywords and context, Robot Framework's actual syntax is, for the purposes of our project, irrelevant. Exceptions to this include the question of which characters of a standard "ASCII" format are syntactically relevant, and its use of "significant indentation".

3.3. Monaco Editor

The Monaco Editor is a code editor with support for features such as "IntelliSense" and "Syntax Colorization", which also makes up the core of the commonly used "VS Code" application.[9]

Our project makes use of its "Syntax Colorization" feature, which we instead refer to as "Syntax Highlighting" in this thesis.

3.4. PostgreSQL

PostgreSQL is an "ACID"-compliant object relational database management system which uses an extended version of the "SQL" language.[10]

PostgreSQL was our choice for our “Mockup Database” system, which is needed to allow for running our project independently from the proprietary “TestRail” test management software.

Part II.

Keywords [F, X, A]

4. Introduction to Keywords [F]

In programming, a "keyword" is a predefined word with a reserved meaning. Keywords can generally not be used for any other purpose, and they make up a programming language's syntax.[11]

In our Script Assembler, we use custom keywords in a similar way, although it may be difficult to define whether or not it is useful to refer to our parser's syntax as a "programming language".

In this chapter, the term "whitespace" is defined to refer to any amount of space or tab characters. Here, for convenience, line-breaks are chosen to specifically not fall under said definition.

4.1. Robot files

In the TRP project, the Robot Framework provides the primary scripting language that makes up the tests to be run on KeTop devices.

A Robot file contains so-called "test cases", which are blocks commonly made up of a few lines of code.[8] Despite most parts of the syntax being irrelevant to the Script Assembler's workings, in order to get a feel for the nature of this project it may still be useful to have a look at an example of a Robot script file:

```
1  *** Settings ***
2  Documentation     A test suite for valid login.
3  ...
4  ...               Keywords are imported from the resource file
5  Resource          keywords.resource
6  Default Tags      positive
7
8  *** Test Cases ***
9  Login User with Password
10     Connect to Server
11     Login User            ironman    1234567890
12     Verify Valid Login    Tony Stark
13     [Teardown]    Close Server Connection
14
15 Denied Login with Wrong Password
16     [Tags]    negative
17     Connect to Server
```

```
18      Run Keyword And Expect Error    *Invalid Password    Login User    ironman
            123
19      Verify Unauthorised Access
20      [Teardown]    Close Server Connection
```

[8]

As is made clear from the example above, some lines may be preceded by whitespace. Since the Robot Framework achieves nesting by having lines be preceded by an integer multiple of exactly four spaces certain lines of code may be syntactically erroneous or interpreted differently when that amount of whitespace is changed.[12] This is an example of a programming convention that in this thesis will be referred to as "significant indentation". Python, for instance, works in a similar way, requiring a predefined use of leading whitespace to achieve grouping of statements.[13]

Despite Robot Framework's general ease of use, proficient support for extensibility, and versatile testing capabilities, the control over the specific execution flow is limited.[14] In the context of KEBA Group AG, a single KeTop may need to run multiple Robot script files, and a Robot script file may need to be reused for multiple KeTops. However, since it may be required for a test to be made of a single script file, it might be necessary to "stitch together", or in other words to concatenate, the contents of multiple Robot script files into one.

Without Script Assembler, a user may run into issues where they, for instance, would like a certain piece of code to only be included on KeTops satisfing a certain condition like being of a predefined operating system, or for certain pieces of code to be executed at a time that is either before or after the remaining parts, as the Robot Framework may not provide a useful away to achieve the required flow.

4.2. What Keywords Are

Our project allows for custom keywords, all of which being preceded by the `@` character, to be defined to act as either "options" or "modifiers". In a provided file, these keywords must be defined at the start of the line, optionally preceded by indentation for nesting purposes. Lines containing such valid keywords will not be included in the concatenated output file.

Each of the keywords, no matter if option or modifier, may be defined to either have no arguments, or have optional or required arguments. If set, these arguments may

then consist of any sequence of characters without line-breaks that is prefixed by the keyword with at least one whitespace inbetween.

The difference between options and modifiers lies in that options do not include "body text", while modifiers do. "Body text" is defined to be made up of all lines of text that, when read from top to bottom, succeed the line of the keyword, up until and not including the first line that is found that does not include at least one additional level of indentation than that of the line of the keyword, or up until the end of the file, ignoring all lines that are either empty or are solely made of whitespace. A level of indentation is defined to be made up of either four spaces or one tab character.

To make up for the extra level of indentation used to define the span of the body text, the parsed body text will be stripped of that one line of indentation. Non-keyword lines, and lines that do not belong to the block body of a modifier will simply be ignored by the Script Assembler and will thus be appended into the output Robot file as-is. This means that, if the provided file does not contain anything that may be matched as a keyword, which, in the case of a valid Robot file, may only be the case in rare exceptions, as the `@` character is generally not utilized in Robot files that are to be executed on a KeTop.

4.3. Examples

4.3.1. Options

For instance, take the following code snippet, which includes the option `@NO_KETOP`, which takes no argumetns:

```
1 @NO_KETOP
2 Install MS
```

When the line containing `@NO_KETOP` is encountered, the `no_ketop` flag, for which the concrete behaviour is explained in the following chapter, is enabled for the output script. Since that specific line will not be included in the output script, assuming the snippet above to be the only input to the parser, the output file may look like the following:

```
1 Install MS
```

4.3.2. Modifiers

Another example is using the modifier `@VERSION`:

```
1  Execute on Ketop    echo A
2  @VERSION BIOS > 1.0.1
3      IF    ${SINGLE_MODE} is $False
4          Execute on Ketop    echo B
5      END
6  Execute on Ketop    echo C
```

Here, the line two, which is made up of `@VERSION BIOS > 1.0.1`, represents a boolean condition relating to the version of the KeTop device. The block body of this condition is made up by the lines three to five. According to the specification of the `@VERSION` keyword, that block body will be included only if the condition stated in its argument is satisfied.

This means that, again assuming the snippet above to be the only input to the parser, if the condition `BIOS > 1.0.1` is satisfied, the output may look like the following:

```
1  Execute on Ketop    echo A
2  IF    ${SINGLE_MODE} is $False
3      Execute on Ketop    echo B
4  END
5  Execute on Ketop    echo C
```

Note the removal of one level of indentation of what used to be the block body. If, instead, the case of the condition is not satisfied, the output may instead be:

```
1  Execute on Ketop    echo A
2  Execute on Ketop    echo C
```

4.4. Recursion

Keywords may also be defined recursively. Recursion is generally defined as being “the process in which a function calls itself directly or indirectly”[15]. In the case of Script Assembler, it refers to the fact that block bodies of modifiers may also include modifiers, of which the block bodies may then also include modifiers, and so forth. To show this possibility, consider the following example:

```
1  @BRACKETSTART
2      @OS WIN10
3          @VERSION BIOS >= 1.7.0
4              Upload Script To KeTop    E2Speed.ps1
```

Here, the block body of `@BRACKETSTART` is made up of lines two to four, of which the block body of `@OS` is made up of lines three and four, of which the block body of `@VERSION` is made up of line four. Despite the order of “which blocks are nested inside which” not making a difference with how all existing modifiers are specified, in general,

the parser will interpret this from the outside to the innermost level of nesting, similar to how nesting is handled by most ordinary programming languages.

This means that, if the above snippet is the only input file that contains the `@bRACKETSTART` keyword, given how that keyword is specified, the part of the output script that is the so-called "bracket start" will be:

```
Upload Script To KeTop    E2Speed.ps1
```

in the case of both of the conditions from the respective specifications of the `@OS` and `@VERSION` keywords and with each of their respective arguments being met. Otherwise, the "bracket start" of the output file will be empty.

4.5. Concatenation and Output

In general, the files get concatenated in the same order that they are given, unless keywords that change that predefined flow are used. Then, to assure the final output Robot file is valid and executable on a KeTop, the following template code will be appended before the output from the entire parsing and concatenation operations:

```
# Template file for HMI Automation
*** Settings ***
Resource          ../../../Robot/Resources/keywords.resource
Resource          ../../../Robot/Resources/variables.resource
Suite Setup       KeTop Suite Setup
Suite Teardown    KeTop Suite Teardown
Test Setup        KeTop Test Setup
Test Teardown     KeTop Test Teardown

*** Test Cases ***
# Here the test cases are inserted
```

Finally, the resulting code will be written to a file which will then be further handled to be run either locally or on a KeTop.

5. The Different Keywords [X]

This section aims to provide a comprehensive list of keywords and their corresponding definitions and uses in the context of the Script Assembler.

5.1. Options

Options are one of the two types of keywords that can be used within a script. They are used to set specific parameters or configurations for the specific script execution, essentially acting as flags that modify certain behaviors when dealing with the script. It doesn't require a value to be assigned to it, so its presence alone is enough to trigger the intended effect. The position of the option keyword within the script is not crucial, so it can be placed anywhere within the script while keeping its intended effect intact.

During the making of the project, one option keyword has been implemented:

5.1.1. Option: NO_KETOP

The NO_KETOP option keyword can be used by typing the following line anywhere within the script:

```
@NO_KETOP
```

If this option keyword is present, the "no_ketop" flag will be set for the entire script. This ensures that certain components that check if a KeTop device is connected are disabled, allowing the script to run without the prerequisite of a KeTop device. This is most useful for specific test scripts that have to be run without a KeTop, like installation scripts.

5.2. Modifiers

Modifiers are the second type of keywords that can be used within a test script. They differ from options in that they require a body, specified by the presence of indentation, to function properly. The body of a modifier contains further lines of the test script, the modifier keyword itself defines under what conditions the body should be included in the final script. Generally speaking, modifiers are used to either conditionally include or exclude certain parts of the script, or set them in a specific order.

Bodies of modifiers can also contain other modifiers, allowing the creation of complex and more specific test scripts.

During the making of the project, the following modifier keywords have been implemented:

- VERSION
- OS
- BRACKETSTART and BRACKETEND
- PREAMBLE

5.2.1. Modifier: VERSION

The VERSION modifier keyword can be used following the syntax:

```
@VERSION <component> <comparison> <version>
    <body>
```

Where:

- **<component>** is the label of the component that is being checked, for example "BIOS".
- **<comparison>** is a comparison operator, such as ">", "<", ">=", "<=", "==" or "!=".
- **<version>** is the version of the component that the test script is being run on, in the format of "X.Y.Z" where X, Y and Z are integers.

If the version modifier keyword is present, the body of the modifier will only be included in the final script if the condition concerning the component version is met. This allows

for the creation of test scripts that behave differently based on the component version of the device.

5.2.2. Modifier: OS

The OS modifier keyword can be used following the syntax:

```
@OS <os>
    <body>
```

Where:

- **<os>** is the label of the operating system that is being checked, for example "WIN10".

If the OS modifier keyword is present, the body of the modifier will only be included in the final script if the device's operating system matches the specified system.

5.2.3. Modifier: BRACKETSTART and BRACKETEND

The BRACKETSTART and BRACKETEND modifier keywords do not require any parameters, only the keyword itself and its body:

```
@BRACKETSTART
    <body>


@BRACKETEND
    <body>
```

If a script containing a BRACKETSTART or BRACKETEND modifier keyword is being run together with other test scripts, the body of these modifiers will be run before and after the content of all other test scripts respectively. If a batch of test scripts contains multiple scripts containing these modifiers, the bodies of the modifiers will be run in the order they appear in the batch, meaning that the body of the first BRACKETSTART modifier will be run before the body of the second BRACKETSTART modifier. The same principle applies to the BRACKETEND modifiers.

Therefore, these modifiers can be used to set up tests that require a certain amount of time to have passed, before the result can be observed. For example, if a test script

is meant to check the behavior of a device after it has been active for 30 minutes, the BRACKETSTART modifier can be used to set up the test, and the BRACKETEND modifier can be used to check the results after the 30 minutes have passed. In the meantime, other test scripts can be run between the two "bracket tests", allowing for a more efficient use of time when running a batch of test scripts.

5.2.4. Modifier: PREAMBLE

The PREAMBLE modifier keyword can be used following the syntax:

```
@PREAMBLE <index>
    <body>
```

Where:

- **<index>** is an integer that specifies the order in which the preambles are run, with 1 being the first preamble to be run.

When one or multiple test scripts containing a PREAMBLE modifier keyword is being run together with other test scripts, the body of the PREAMBLE keyword will be run before the content of all other test scripts, including lines in the body of a BRACKETSTART modifier. In the case of multiple PREAMBLE modifiers being present in the batch of test scripts, the bodies will be run in the order of their specified index.

The PREAMBLE modifier can be used to write scripts that test certain installation processes and their successful completion before the rest of the test scripts are run.

5.3. Visual Example

Consider the following diagram:

Here, we can see how some specific test scripts are arranged in the final script.

1. The bodies of the PREAMBLE modifiers are ordered first if present. In the case of multiple PREAMBLE scripts being present, the specified index is used to order them accordingly. As seen in the diagram, leaving out certain indices doesn't lead to any issues, the ordering occurs as intended.

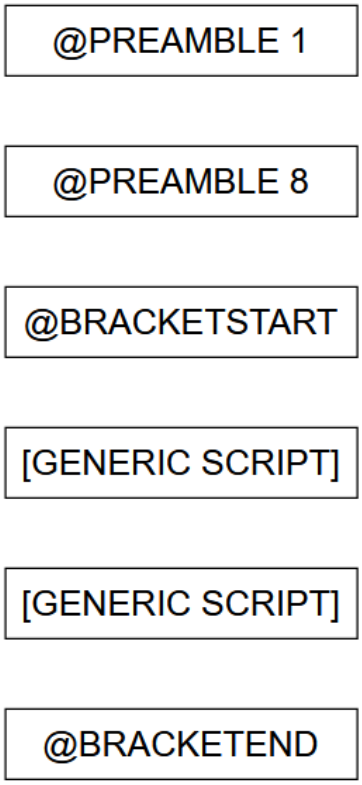

Figure 1.: Simple Diagram depicting the order scripts with a certain modifier keyword are run in.

2. The bodies of the BRACKETSTART modifiers are ordered next if present. In the case of multiple BRACKETSTART scripts being present, the order they appear in the batch is maintained.

3. At this point, the content of all test scripts not containing any order-modifying keywords are placed. Their order is determined by the order they appear in the batch.

4. Lastly, the bodies of the BRACKETEND modifiers are ordered if present. In the case of multiple BRACKETEND scripts being present, they follow the same principle as the BRACKETSTART modifiers, meaning that the order they appear in the batch is maintained.

6. Why This Syntax? [A]

Part III.

EBNF [A, F]

7. Overview of EBNF [A]

7.1. Motivation for Formal Syntax Descriptions

To be able to run tests automated on every kind of KeTop, there is a need to modify test scripts depending on the given environment. That is why an extended Robot Framework language was needed. The Extended Backus-Naur Form (EBNF) was chosen for this, to make this new language easily extendable and deterministic to implement if adopted by other developers.

In practice, in the field of computer science, it is clear that a precise description of a language is the most important foundation. It is therefore essential that applications, communication protocols or programs function according to clear syntactic rules so that they are always interpreted in the same way by humans and machines. Natural language provides many different ways of describing something. There are ambiguities in language, and these can lead to errors in interpretation. These findings are not new. They have been known for a long time. Nevertheless, there is still no common standard that everyone adheres to, especially when languages have been implemented independently by different developers (cf. [16, p. V]).

Syntactic metalanguages exist to solve this problem. The goal of a metalanguage is to describe another language. This means there is a difference between the language that is described and the language used to describe it.

In programming, a metalanguage is a language used to describe the syntax and structure of another language, called the object language. For example, BNF and EBNF are metalanguages used to define programming languages such as ALGOL. The structure of sentences and expressions in a metalanguage can be defined using a so-called metasyntax. For example, one could indicate that the word “noun” can itself function as a noun in a sentence by writing: “noun” is a `<noun>`. (cf. [17])

7.2. The Backus-Naur Form (BNF)

One of the most important metalanguages for programming has been BNF (the so-called Backus-Naur Form). BNF started with ALGOL 60 in 1960. ALGOL 60 needed a clear, formal way to describe its syntax. John Backus and Peter Naur therefore created BNF. It made it possible to define language rules precisely. This was the first time syntax was described formally. The history of ALGOL 60 is crucial because it gave BNF its first real use. BNF became the foundation for defining many programming languages. It is still essential in compilers and language design today. (cf. [18, p. 3])

As BNF was first used for ALGOL 60 and became the model for many later languages, over time, people added changes or extensions, which created some confusion. Despite this, BNF showed the importance of clear, formal syntax.

Therefore, Extended BNF was later developed from BNF and included the most common extensions. The large amount of variants motivated the standardization effort that resulted in ISO/IEC 14977. (cf. [16, p. V])

7.3. The Evolution to Extended BNF (EBNF)

For early programming languages such as ALGOL 60, BNF was adequate. However, the increased complexity of modern languages required a more systematic notation to describe their syntax effectively.

Therefore, the so-called Extended Backus-Naur Form (EBNF) was introduced. The inventor of Extended BNF - EBNF, is Niklaus Wirth, a Swiss computer scientist. EBNF is based on BNF but it adds more elements.

> “One cannot fail to notice the lack of ‘common denominators.”

Niklaus Wirth stated this in his first article in the journal “Communications of the ACM” in 1977 about EBNF. [19]

He explained that there is a lack of common standards or unified fundamental principles across different language definitions and notational forms. Each language definition tends to use slightly different variants of the Backus-Naur form based notation. There is no generally accepted standard, and the differences are often small but unnecessarily

varied. He stated that a more uniform notation would improve scientific work and comparison. EBNF is his solution for this problem.

As mentioned EBNF is clearly based on BNF, but it adds more elements. EBNF can define exact numbers of elements, describe exceptions, add comments, use flexible rule names, and allow extensions. This makes grammar definitions shorter and it is easier to read them. But EBNF also has restrictions. It can only handle languages with elements in order and is not able to describe very complex grammar. (cf. [16, pp. VII-VIII])

The main disadvantage of BNF according to ISO/IEC 14977 was:

> "Backus-Naur Form (used in ALGOL 60) has problems if the metasymbols < > ::= occur in the language being defined. Some common forms of construction (e.g. comments) cannot be expressed naturally; other constructions (e.g. repetition) are long winded." [16, p. VII]

The added advantages, but also the limitations of EBNF, will be discussed in more detail in later chapters.

7.4. Wirth's Approach to Syntax Definition

Having established the historical need for a unified standard in the previous section, it is now necessary to examine the concrete syntactic rules proposed by Niklaus Wirth in 1977. In his article published in the journal Communications of the ACM (CACM), Wirth addressed the existing problems of syntactic notation directly. His contribution marked an important step in the formal development of language definitions.

Wirth did not intend to introduce a completely new theoretical concept. Instead, his objective was to improve the existing Backus-Naur Form in a systematic way. BNF had already proven to be useful. However, it contained ambiguities and practical limitations.

The notation presented in his article was therefore designed to be more practical and clearer in structure. It focused on improving readability and standardization rather than expanding theoretical expressive power. The goal was not innovation for its own sake. The goal was clarity, consistency, and easier application in real technical environments.

Wirth listed the following properties to extend BNF:

1. The notation clearly separates meta symbols, terminal symbols, and nonterminal symbols.

   Example: `syntax = "production".`

   Here, `syntax` is a nonterminal symbol. `"production"` is a terminal symbol. The equals sign and the dot are structural meta symbols.

2. Symbols used for structure can also be used as normal language symbols. This avoids artificial restrictions.

   Example: If quotation marks are used as meta symbols, they can still appear inside text if written twice, like `""` in many programming languages.

3. The notation contains a simple way to express repetition. This reduces the need for recursion in simple cases.

4. The notation avoids explicit representation of the empty string, such as by using `<empty>` or ε.

5. The notation is based on the ASCII character set. This makes it easy to use in most technical environments.

(cf. [19])

In his article, he showed the power of EBNF by defining its own syntax as an example (see figure 2):

> "This meta language can therefore conveniently be used to define its own syntax, which may serve here as an example of its use. The word identifier is used to denote nonterminal symbol, and literal stands for terminal symbol. For brevity, identifier and character are not defined in further detail." [19]

```
syntax     = {production}.
production = identifier "=" expression ".".
expression = term {"|" term}.
term       = factor {factor}.
factor     = identifier | literal | "(" expression ")" |
             "[" expression "]" | "{" expression "}".
literal    = """ character {character} """.
```

Figure 2.: Definition of EBNF in EBNF in "What Can We Do about the Unnecessary Diversity of Notation for Syntactic Definitions?" from N. Wirth [19]

7.5. Core Syntactic Extensions

The following overview focuses more in detail on the differences in EBNF and of the Extended Backus-Naur Form (EBNF) as described in Concepts of Programming Languages by Robert W. Sebesta. (cf. [20, pp. 149-151])

It is essential to note that these extensions do not enhance the descriptive power of BNF. The extensions are made to increase the readability and writability of formal grammars. BNF is theoretically sufficient for describing context-free languages, but EBNF provides a more concise notation for complex structures.

7.5.1. Core Syntactic Extensions in EBNF in detail

Sebesta identifies three primary extensions common to most versions of EBNF:

- **Optionality** (`[...]`):
  Brackets are used to denote a part of a right-hand side (RHS) that may occur once or not at all.
  Example: A C if-else statement can be described as: `<if_stmt>` → `if (<logic_expr>) <stmt> [else <stmt>]`.
  Instead one uses square brackets to make something optional. For example `["a"]` means “a” or empty.

- **Repetition** (`{...}`):
  Braces indicate that the enclosed part can be repeated any number of times, including zero. This replaces the need for recursion when describing simple lists.
  Example: A list of identifiers separated by commas: `<ident_list>` → `<identifier> {, <identifier>}`.
  Example: `{a}` means a, aa, aaa, and so on.

- **Multiple-Choice Options** (`(...)` and `|`):
  When one element must be selected from a group, the options are grouped in parentheses and separated by the OR operator.
  Example: Defining a term with different operators: `<term>` → `<term> (* | / | %) <factor>`.

In this notation, brackets, braces, and parentheses serve as metasymbols (notational tools) rather than terminal symbols of the language being described. (cf. [20, pp. 149-151])

7.5.2. Further Syntactic Extensions in EBNF based on ISO/IEC 14977

However, there are more differences between BNF and EBNF in the ISO/IEC 14977 definition: (cf. [16, p. VII])

- **Defining an explicit number of items:** In some cases it is needed to describe, that e.g., a field must have an exact length. In BNF, this is difficult. Often, the programmer has explained it then informally in English. Now EBNF solves this by allowing an integer followed by a repetition-symbol (`*`). With this solution, the programmer can specify the exact number of items. This makes rules much shorter.
- **Exceptions:** Sometimes it would be easier to define a rule by specifying what is not allowed. But BNF cannot easily say "any character except a quote." EBNF has a solution for that. EBNF uses an except-symbol (`-`) to define rules by exclusion. This makes definitions shorter and easier. Therefore, for example, a programmer can define a group of symbols and then subtract specific exceptional cases, such as "any character except a quote".
- **Formal comments:** Comments are an important feature, that BNF is not offering. EBNF provides a formal way to add comments using `(*` and `*)`. And these notes do not change the machine's logic.
- **Flexible names:** In BNF, names for categories often need brackets. EBNF uses an explicit comma (`,`) as a so-called concatenate symbol to link parts of a rule. This allows category names to use multiple words without brackets, meta-identifiers no longer have to consist of a single word or be enclosed in angle brackets. It also ensures that the layout of the syntax like spaces or line breaks has no effect on the defined language.
- **Extensions:** Programmers can add their own features to EBNF. These are called "special sequences" and are placed between question marks (`?`). This makes the start and end of any extension very easy to find. Extensions were used several

times in earlier versions of the solution presented in this thesis, but were eliminated in the final version because there was no longer a need for them.

7.5.3. Comparative Notation BNF and EBNF

While classical BNF is theoretically sufficient for describing any context-free language, it is often complicated and repetitive in practice. The main cause therefore is, that it relies, as described in the previous chapter, exclusively on recursion to express lists and optional elements. In the following illustration one can see the advantage of EBNF introducing specialized meta symbols, such as braces, brackets, and parentheses. [20, p. 153]

BNF and EBNF Versions of an Expression Grammar

BNF:

```
<expr> → <expr> + <term>
       | <expr> - <term>
       | <term>
<term> → <term> * <factor>
       | <term> / <factor>
       | <factor>
<factor> → <exp> ** <factor>
         | <exp>
<exp> → (<expr>)
      | id
```

EBNF:

```
<expr> → <term> {(+ | -) <term>}
<term> → <factor> {(* | /) <factor>}
<factor> → <exp> { ** <exp>}
<exp> → (<expr>)
      | id
```

Figure 3.: Comparison between BNF and EBNF by Sebesta [20, p. 153]

As demonstrated in the example in figure 3, the structural differences lead to a significant reduction in the number of rules required:

- **Repetition via Braces in this example:** In the BNF version of the expression grammar, an `<expr>` must be defined recursively to handle a sequence of additions or subtractions. EBNF replaces this recursion with braces `{...}`, which denote that the enclosed part can be repeated indefinitely or left out entirely.
- **Multiple-Choice Options in this Example:** The example uses parentheses `(...)` combined with the OR operator `|` to group choices. This for example allows the EBNF version to handle both addition and subtraction in a single line. In BNF it requires a more complex approach.

7.5.4. Modern Variations and Standardization

While an official standard exists (ISO/IEC 14977:1996), it is rarely used in practice. There have several variations of EBNF emerged in recent years, which deviate from the standardized syntax, such as using a colon instead of an equal sign or a dot instead of a semicolon. Also the project described in this thesis partially deviates from ISO standards due to using the EBNF dialect of the Lark Python library.

7.6. ISO/IEC 14977 Standards for Assembler Logic

While the previous chapters describe the advantages of EBNF in terms of readability and compactness, the ISO definition has an additional function. The ISO standard ISO/IEC 14977 also defines strict technical rules.

These are very important for error-free processing by an automated tool such as a script assembler, as they ensure that the grammar definition remains mathematically unambiguous.

7.6.1. Operator Precedence

For every EBNF grammar, the strictly defined hierarchy of meta-symbols is essential, as it prevents ambiguities in the rules. The order of the symbols is clearly defined in the ISO standard without any leeway and is specified as follows: (the strongest is the repetition symbol. This takes precedence, followed by the exclude symbol, and so on).

1. Repetition symbol (*): For specifying exact numbers of repetitions.
2. Except symbol (-): For defining sets by exclusion.
3. Concatenate symbol (,): For explicitly linking consecutive elements.
4. Definition separator symbol (|): For separating alternatives.
5. Defining symbol (=): Formally assigns an expression to a nonterminal.
6. Terminator symbol (;): To clearly mark the end of a production rule.

(cf. [16, p. 1])

7.6.2. Layout Neutrality Through Gap Separators

Ideally, a script assembler should be able to process scripts regardless of their visual formatting. The ISO standard is helpful here, as it defines so-called "gap separators" (such as spaces, tabs, or line breaks) (cf. [16, pp. 5]). These characters have no formal meaning outside of terminal strings. This has the significant advantage of allowing for flexible grammar design without affecting the assembler's logic. Since gap separators and comments have no formal effect on the defined language, the script assembler can ignore these characters during analysis, which, for example, increases its robustness against different programming styles.

In summary, several advantages arise. The grammar definition remains mathematically consistent for automated tools. Furthermore, it is beneficial that the grammar can be made more readable and understandable for human readers through formatting and annotations.

7.6.3. Syntactic Exceptions

As mentioned earlier, the Except symbol (-) allows the assembler to define specific instruction sets. An example of this would be "all characters except a semicolon." However, the ISO standard also imposes very clear technical limitations on this function, as otherwise paradoxes could arise:

> "If a syntactic exception is permitted to be an arbitrary syntactic factor, Extended BNF could define a wider class of languages than the context-free grammars, including attempts which lead to Russell-like paradoxes... Such a license is undesirable and the form of a syntactic exception is therefore restricted to cases that can be proved to be safe." [16, p. 2]

A good explanation for the Russell Paradox, formulated by Bertrand Russell, is the so-called barber's example: In a village, the barber shaves precisely all the men who don't shave themselves—and only these men. The question of whether the barber shaves himself leads directly to a contradiction: If he shaves himself, he shouldn't, by definition; if he doesn't shave himself, he should. (cf. [21])

This example illustrates that such self-referential definitions can lead to logical inconsistencies in formal systems.

7.6.4. Alternative Representations for System Compatibility

To guarantee that a grammar functions on different systems, the ISO standard allows the substitution of certain special characters if they are missing. For example, the characters `(:` and `:)` can replace `{` and `}`, or a period (`.`) can replace a semicolon at the end of a rule. This ensures that the grammar remains readable and functional on different platforms without losing its original structure. (cf. [16, pp. 5])

7.6.5. Special Sequences as a Formal Interface for Assembler Extensions

Although special sequences were already described in the previous section as a general extension option, they are listed again here to specifically clarify their "technical" role as placeholders. In the logic of a script assembler, they serve as a standardized mechanism to mark parts of the language that cannot be defined by EBNF itself (e.g., specific character sets). (cf. [16, p. VII])

By enclosing them in question marks (`?...?`), the grammar remains formally valid for the assembler tool. In this work, they thus can provided the flexibility for possible future functional extensions without violating the fundamental syntactic rules of the ISO standard. (cf. [16, p. 3])

7.6.6. Evaluating the Standard for our Script Assembler

After introducing the theoretical foundations of EBNF, it is necessary to evaluate how the ISO standard was applied in the context of the script assembler. The goal of the project was not to implement a complete parser for the Robot Framework. Instead, the focus was on building a preprocessor for specific control tags such as `@OS`. For this reason, the resulting grammar is not a strict implementation of the ISO standard. It represents a pragmatic combination of formal rules and practical requirements.

A central aspect of this evaluation is the concept of layout neutrality. The ISO standard defines spaces, tabs, and line breaks as so-called gap separators. These elements have no formal meaning and can be ignored by the parser. This principle improves flexibility and readability. However, it could not be applied completely in this project. Inline

spaces and tabs between control tokens are treated as insignificant and are therefore ignored. This allows test scripts to remain flexible in their formatting.

Line breaks, however, require a different treatment. Robot Framework scripts are structured line by line. This means that line breaks carry semantic meaning. They define the structure of the script. Therefore, line breaks could not be treated as layout-neutral elements. Instead, they were explicitly defined as structural tokens in the grammar. This is a necessary deviation from the ISO specification to ensure correct processing of the input.

Further deviations from the standard were made in the handling of syntactic constraints. The ISO standard introduces syntactic exceptions to define rules by exclusion. However, this mechanism can become complex and difficult to understand. In some cases, it can even lead to problematic definitions. For this reason, syntactic exceptions were not used in the assembler.

Instead, the implementation relies on regular expressions. Regex provides a practical way to capture and process text patterns. It allows the parser to treat sections of Robot Framework code as plain content. The internal structure of these sections is not analyzed. This simplifies the grammar and reduces its complexity. As a result, the grammar becomes easier to read and maintain.

8. Approaches to EBNF [A]

9. A Problem with EBNF [F]

10. Our EBNF [F]

Part IV.

Implementation [A, F, X]

11. Preprocessor [A]

12. Parser [F, A]

13. Syntax Highlighting [A]

14. Extensibility [F]

15. Database [X]

Besides the extension of the Script Assembler's capabilities and the creation of a new EBNF language that makes the software more extendable, another main focus of the project is the creation of a mockup of the TestRail database. This mockup is not only made for testing purposes, but allows the user to work independently of the TestRail instance. The mockup is designed to be as close to the real TestRail database as possible, allowing a seamless switch between the two instances. This means that all functionalities that are available in the real TestRail database are also available in the new mockup.

This section aims to give an overview of the different types of databases, the structure and functionalities of the TestRail database specifically, the resulting design and implementation of the mockup database, and the way the database switch is finally implemented.

15.1. Types of Databases

Depending on the requirements of a given project, different types of databases are suitable for different use cases. It is important to analyze what qualities are needed and which qualities are negligible to find the best fitting solution.

The most important qualities of a database type can be expressed in the following categories:

- **Data Model:**
  The data model of a certain database type describes how the data is structured and stored. This can be in the form of tables, documents, graphs or other structures. The data model has a notable impact on aspects like performance and scalability.
- **Storage Architecture:**
  The storage architecture of a database type not only describes how the data is

stored, but also how it may be accessed, managed and distributed. This can be in the form of a single server, a cluster of servers or even a distributed network of servers across the globe.

- **Query Model:**
  The query model of a database type describes the means of accessing the data. This can occur in the form of a specifically designed query language, an API or other methods. This has an undeniable impact on the ease of use of the database and how the user writes and optimizes queries.

Based on these qualities, various databases types have been developed, each with their own unique advantages and disadvantages. Some of the most common types of databases include but are not limited to relational databases, object-oriented databases and NoSQL databases, which can be further categorized by the data model they use, such as document stores, key-value stores, column-family stores and graph databases. (cf. [22])

15.2. Relational Databases

For the purpose of this project, we will take a closer look at relational databases, as this is the type of database that TestRail uses.

15.2.1. Structure of Relational Databases

Relational databases are based on the relational model, which organizes its data into tables, consisting of rows and columns. Each table represents a specific entity and each row in the table represents an instance of that entity. The columns in the table represent the attributes of the entity.

Relationships between these entities can be established through the use of keys. A primary key is a unique identifier for each instance of an entity, while a foreign key is a reference to the primary key of another entity. Meaningful relationships between entities can be built with the usage of these keys, allowing complex queries to be executed across multiple linked entities.

Grouping in queries allows for the aggregation of data based on specific attributes, while joins allow for the combination of data from multiple tables based on the established relationships.

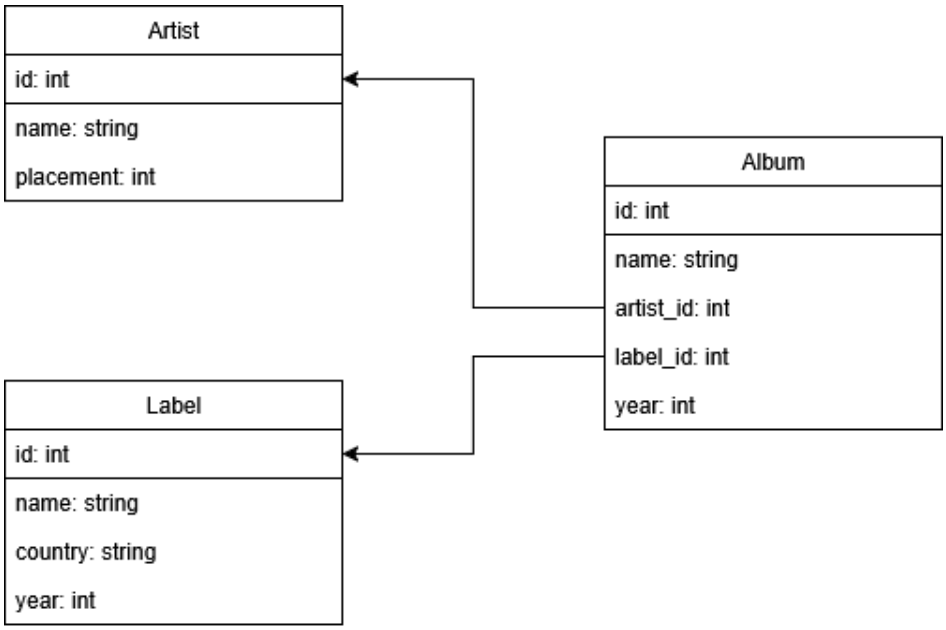

Figure 4.: Simple Diagram showcasing the relationship between three entities in a relational database.

In figure 4, we can see a simplified example of a relational database structure. The three entities "Artist", "Label" and "Album" are represented as tables, with their respective attributes as columns. The relationships between the entities are established through the usage of primary and foreign keys, allowing for queries to be executed across multiple linked entities. For example, we can query for all albums by a specific artist or a specific label.

15.2.2. ACID

ACID is an acronym that stands for the four key properties of a safe database transaction:

- **Atomicity:**
  Atomicity ensures that a transaction, which is often a sequence of multiple operations, is treated as a single unit. This means that either all operations within a given transaction are executed successfully, or none of them are. If any operation of the transaction fails due to an error, a power outage or any other issue, the entire transaction is rolled back to its previous state before the

transaction began. This guarantees that the database remains consistent and prevents data corruption or loss that could occur if only part of the transaction were to be executed.

- **Consistency:**
  Consistency ensures that the database can only be brought from one valid consistent state to another, while following all defined rules, constraints and triggers. Invalid transactions that violate any of the defined rules are rolled back accordingly, ensuring that the database remains consistent.

- **Isolation:**
  Isolation ensures that concurrent transactions have no impact on each other. This means that the effects of a transaction in progress are not visible to other transactions until it completes successfully. This prevents a number of issues that would otherwise arise from concurrent transactions, like dirty reads, where one transaction reads data that has been modified by another transaction but not yet fully committed, potentially leading to incorrect results.

- **Durability:**
  Durability ensures that once a transaction has been successfully completed and committed, the state of the database persists even in the case of a crash, power failure or any other issue.

Databases that adhere to these properties are considered to be reliable and safe for use in critical environments, as they guarantee that the data not only remains consistent, but also that it is not lost or corrupted due to unforseen events. (cf. [23])

15.2.3. Advantages and Disadvantages of Relational Databases

One of the main strengths of relational databases is their ability to handle long and complex queries across multiple linked entities with the usage of foreign keys and joins. The consistency and integrity of the data is guaranteed by the ACID properties. On top of that, relational databases are some of the most widely used and well-known types of databases, which means that there are a lot of resources and a large community available for support and development.

However, relational databases can struggle with scalability and performance when dealing with large volumes of data or high traffic. The fixed schema of relational

databases can also be a disadvantage, as it can make it difficult to adapt to changing requirements or to handle unstructured data. Additionally, the execution of complex queries can lead to performance issues on larger datasets.

15.3. The TestRail Database

TestRail is a web-based test management tool that provides a wide range of features for managing and organizing test cases, test runs and test results. The relational database in the background of TestRail is used to store all of the data related to these features. The user can interact with the database directly by using the TestRail API, although the database itself is not directly accessible to the user.

Thus, for the creation of the mockup database, the structure and functionalities of the TestRail database have to be studied and analyzed based on the API documentation and the structure of its response data. While the mockup database doesn't have to be an exact replica of the TestRail database, the functionalities used by the Script Assembler need to deliver responses with identical structure to ensure the switch between databases can occur without any issues.

In order to achieve this, the most important entities and their relationships have to be identified.

15.3.1. Test Cases

Test cases can be seen as the most fundamental entity in the TestRail system, as they represent the individual tests that are to be executed. Test cases have a number of attributes, such as a title, description, type and the test steps that need to be executed. Test cases are furthermore linked to test suites and sections.

15.3.2. Projects, Suites and Sections

Projects are the highest level of organization in TestRail, representing a specific testing effort or initiative. Each project can contain multiple test suites, which are collections of test cases. The test cases of Suites can further be organized into sections, allowing for more specific grouping and categorization. These relationships are an essential part of the database structure that needs to be replicated.

15.3.3. Test Runs

A test run represents a specific execution of test cases. Test runs are linked to test cases and contain information about the execution, such as the status of each case, the tester who executed the run and any relevant comments or other metadata. Test runs make the tracking and management of test executions possible. Importantly, a test run can only be created for test cases that are contained in one singular test suite.

15.3.4. Test Plans

Test plans are used to organize and manage test runs. TestRail allows the user to group multiple test runs into a test plan, which alleviates the need to manage each run individually.

15.3.5. Test Results

Test results represent the outcome of a test case within a test run. Test results mainly consist of a status, which can contain, but is not limited to:

- Passed
- Failed
- Untested
- Retest
- Blocked

15.3.6. Users

Users represent the individuals who interact with the TestRail system. Users can have different roles and permissions within the system. They can be assigned to test runs, test plans or other entities as creators. With the user entity it is possible to track who is responsible for which run and plan.

15.4. PostgreSQL

For the implementation of the mockup database, PostgreSQL was chosen as the database management system, as it is an open-source database that is not only widely used but also well-known for its reliability and performance.

PostgreSQL is not only a database management system that the team is familiar with, but it also provides all the necessary features and capabilites needed for the implementation of this part of the project. It supports the relational data model, which is needed for replacing the TestRail database, and it guarantees the reliability and consistency of its data with the ACID properties.

The open-source management tool "pgAdmin" is used for the administration of the PostgreSQL database, as it provides an easily navigable user interface for management, query execution and other administrative tasks.

15.5. First Approaches

Early into the development of the database, two main approaches were considered for how the database should interact with the backend of the main system.

One approach was to create an API that would allow the main system to interact with the database through API calls. This would have resulted in a clear separation between the database and the main system.

The other approach was to work in the preexisting backend to connect to the database directly. This approach was ultimately chosen, as it not only makes the switch between the two databases simpler, but also integrates the functionality of the new database directly into the main system. This approach was further endorsed by the supervisor of the project, as it helps the system to eventually transition away from the TestRail database and towards the new mockup database, which allows for more independence and expandability in the future.

The python library "psycopg" is used for the connection between the main system and the PostgreSQL database. This library provides a simple and efficient way to connect to the PostgreSQL database and execute various commands and queries. It also provides support for connection pooling, which can help to improve the performance of the database interactions.

15.6. Database Connections in the System

The following section aims to give a brief look into the structure and functionalities of not only the existing TestRail database connection, but also the new PostgreSQL mockup database. The goal is to show how the new database is designed to replicate the implementations of the TestRail connection, while also highlighting clear differences on a technical level.

15.6.1. Existing TestRail Functionality

The existing funtionalities concerning the TestRail connection is contained within the TRSession class, which is responsible for all interactions with the database. This class contains functions for retrieving and creating test cases, runs, plans and results, as well as other relevant data.

The communication with the TestRail database is achieved through the usage of the TestRail API, which provides all necessary reading and writing functionalities.

```python
def get_case_title(self, case_id: int, safe_name: bool = False) -> str | None:
    '''
    Returns the Name of a test case by a given test case id
    Parameters:
        case_id (int):    Name (Title) of the testcase. (partial match with contains)
        safe_name (bool): if true: delete the ":" character in the title for file system compliance
    Returns:
        (string):  The ID of the test case in section: section_name containing: testcase_name
        None: no test case was found
    '''

    response = self.do_request(f"get_case/{case_id}", TRConst.REQUEST_TYPE_GET)
    if not response:
        return None
    if len(response) <= 0:
        self.log.warning(f"could not find a result for test case: {case_id}")
        return None
    if not safe_name:
        return response['title']
    return response['title'].replace(":", "")
```

Figure 5.: Code snippet of the get_case_title function within the TRSession class.

In figure 5, one of the functions of the TRSession class can be seen. This function is responsible for retrieving the title of a test case based on its ID. Using the ID of the wanted test case, a GET request is sent to the TestRail API, which then returns the relevant data. The function then processes the response data and returns the title.

All functions of the TRSession class follow a similar structure, with the type of interaction with the API being the main difference. For example, functions that create new entities

in the database use POST requests and send the relevant data in the body of the request.

15.6.2. DBSession

The DBSession class is designed to replicate all used functionalities of the exisitng TRSession class, while replacing the communication with the TestRail API with a direct connection to the PostgreSQL database, using the "psycopg" python library. It is strictly necessary that the functions of the DBSession have the same name, parameters and return values as the functions of the TRSession class, in order to avoid major conflicts when switching between the two database connections.

```python
def get_case_title(self, case_id: int, safe_name: bool = False) -> str | None:
    '''
    Returns the Name of a test case by a given test case id
    Parameters:
        case_id (int):    Name (Title) of the testcase. (partial match with contains)
        safe_name (bool): if true: delete the ":" character in the title for file system compliance
    Returns:
        (str | None): on success: title of found case - else: None
    '''

    if case_id == None:
        print("Did not get a case_id")
        return None

    if case_id < 0:
        print(f"Invalid value for case_id: {case_id}")
        return None

    self.cursor.execute("SELECT title FROM test_cases WHERE id = %s", (case_id,))

    result = self.cursor.fetchone()

    if result != None:
        return str(result[0])

    return None
```

Figure 6.: Code snippet of the get_case_title function within the DBSession class.

In figure 6, the same function as in figure 5 can be seen, but this time within the DBSession class. Thus, while the parameters and return value of the function are identical to the one in the TRSession class, the implementation is different.

Instead of interacting with an API, this function uses a connection to the PostgreSQL database to execute a SQL query that retrieves the title of the specified test case.

15.7. Integration of the DBSession Class

Since the DBSession class replicates all essential functions of the TRSession class, the switch between database usages can be achieved easily.

Any occurrence of the TRSession class within the backend of the system is now denoted as a TRSession or DBSession, using a pipe symbol. This means that at system startup, the appropriate session class has to be identified and initialized accordingly.

This way, the TRSession functionalities remain unaltered, while the possibility of using the DBSession is a new feature that can be easily switched on or off.

15.7.1. Switching Between Databases

The switch between the two databases is implemented in a way that allows the user to easily and definitively choose which database should be used at the execution of the python script. This is achieved by using a command line argument that can be passed when executing the script. The argument is then processed and the appropriate database instance is initialized based on if the argument is present or not.

```
python ./broker.py
```

Here, the standard TestRail database is used, as no argument is passed to the script.

```
python ./broker.py -db
# OR
python ./broker.py --database
```

Here, the PostgreSQL mockup database is used, as the argument is passed to the script. The argument can either be a short version, or a long verion, as both are processed in the same way and lead to the same result.

15.8. The PostgreSQL Database

15.9. Potential Future Steps

15.9.1. Easier Creation of Test Cases

15.9.2. Encryption

1

References

[1] Python Software Foundation, “What is Python? Executive Summary,” retrieved 2026-02-21. [Online]. Available: https://www.python.org/doc/essays/blurb/

[2] Parewa Labs Pvt. Ltd., “Python Modules (With Examples),” retrieved 2026-03-01. [Online]. Available: https://www.programiz.com/python-programming/modules

[3] Python Software Foundation, “Installing Python Modules,” retrieved 2026-03-01. [Online]. Available: https://docs.python.org/3/installing/index.html

[4] Erez Shinan, “Lark documentation,” retrieved 2026-03-01. [Online]. Available: https://lark-parser.readthedocs.io/en/latest/

[5] Raphaël Barrois, “semantic-version · PyPI,” retrieved 2026-03-01. [Online]. Available: https://pypi.org/project/semantic-version/

[6] Tom Preston-Werner, “Semantic Versioning 2.0.0,” retrieved 2026-03-01. [Online]. Available: https://semver.org/

[7] Daniele Varrazzo, “The Psycopg 3 project,” retrieved 2026-03-01. [Online]. Available: https://www.psycopg.org/psycopg3/

[8] Robot Framework Foundation, “Robot Framework,” retrieved 2026-03-01. [Online]. Available: https://robotframework.org/

[9] Microsoft, “Monaco Editor,” retrieved 2026-03-01. [Online]. Available: https://microsoft.github.io/monaco-editor/

[10] The PostgreSQL Global Development Group, “PostgreSQL: About,” retrieved 2026-03-01. [Online]. Available: https://www.postgresql.org/about/

[11] Nikhil Garg, “What is a Keyword in Programming?” retrieved 2026-03-01. [Online]. Available: https://www.geeksforgeeks.org/dsa/what-is-keyword-in-programming/

[12] Robot Framework Guides, “Style Guide,” retrieved 2026-03-01. [Online]. Available: https://docs.robotframework.org/docs/style_guide

[13] Python Software Foundation, "Indentation in Python," retrieved 2026-03-01. [Online]. Available: https://docs.python.org/3/reference/lexical_analysis.html#indentation

[14] Elly Johnson, "Pros and Cons of Robot Framework," retrieved 2026-03-01. [Online]. Available: https://testautomationtools.dev/pros-and-cons-of-robot-framework-general-overview/

[15] Sonal Tuteja, "Introduction to Recursion," retrieved 2026-03-02. [Online]. Available: https://www.geeksforgeeks.org/dsa/introduction-to-recursion-2/

[16] ISO/IEC 14977, "Information technology - Syntactic metalanguage - Extended BNF," *International Standard 14977:1996(E)*, vol. 1996, pp. 1–24, 1996. [Online]. Available: http://www.iso.org/iso/catalogue_detail.htm?csnumber=26153

[17] Wikipedia contributors, "Metalanguage," 2026, accessed: 2026-02-23. [Online]. Available: https://en.wikipedia.org/wiki/Metalanguage

[18] H. Mössenböck, *Compiler Construction*, 2025.

[19] N. Wirth, "What Can We Do about the Unnecessary Diversity of Notation for Syntactic Definitions?" vol. 20, no. November, pp. 1–2, 1977.

[20] R. W. Sebesta, *Concepts of Programming Languages ELEVENTH edition.*

[21] Wikipedia contributors, "Barbier Paradoxon," 2026, accessed: 2026-02-23. [Online]. Available: https://de.wikipedia.org/wiki/Barbier-Paradoxon

[22] Vinayak Baranwal, "What are the different types of databases? explained with use cases and architectures," 2025, accessed: 2026-02-28. [Online]. Available: https://www.digitalocean.com/community/conceptual-articles/database-types

[23] Wikipedia contributors, "Acid," 2026, accessed: 2026-02-28. [Online]. Available: https://en.wikipedia.org/wiki/ACID

List of Figures

List of Tables

List of Listings

Appendix